

AI 智能人岗匹配平台

产品需求文档 v2.0 | 2026-05-31

版本	日期	作者	变更说明
v2.0	2026-05-31	AI产品团队	重构为 write-a-prd 标准模板；深度细化技术

Problem Statement

用户面临的核心问题

中国企业希望招聘国际实习生，但面临三重障碍：

语言障碍：收
国际学生看不

国际学生希望来华实习，但同样面临困境：

信息获取：看

平台运营方则面临“鸡生蛋”问题：没有足够的 AI 能力就难以规模化，不规模化又无法摊薄 AI 成本。

当前状态

MVP 已上线运行（岗位发布、简历上传、即时聊天），有日活用户。AI 能力为零，所有匹配依赖人工或简单关键词筛选。

Solution

在不推倒现有 MVP 的前提下，渐进式引入四大 AI 能力：

简历解析：多语言简历 → 结构化数据，让 HR 一眼看懂外文简历

岗位翻译：中

技术策略：RAG + LLM API，无需自训模型，MVP 阶段用 pgvector 零成本起步，向量规模扩大后平滑迁移到 Milvus。

User Stories

A. 简历解析

作为一个 HR，我希望上传一份英文 PDF 简历后，系统自动提取出姓名、邮箱、技能、教育背景、工作经历，以结构化的中文/英文卡片展示，这样我无需阅读长篇外文简历就能快速判断候选人

作为一个
Word、PDF、

B. 岗位翻译

作为一个国际学生，当我浏览实习岗位列表时，希望能看到英文版的 JD，而不只是中文原文，这样我才能准确判断这个岗位是否适合我

作为一个国际
，而不只是字

C. 人岗匹配

作为一个 HR，当我发布新岗位后，希望能立即看到系统中已有简历与该岗位的匹配排序，最匹配的候选人排在最前面

作为一个
HR，我不仅想
侧重点做最终

D. 简历优化建议

作为一个国际学生，当我看到一个感兴趣的岗位但匹配度只有 60% 时，希望能得到具体的简历修改建议，告诉我改哪些地方可以提升匹配度

作为一个国际

E. 系统基础能力

作为一个产品负责人，我希望所有 AI 调用都有 Token 用量和成本的实时监控，避免 API 费用失控

作为一个产品
版本），而不

Implementation Decisions

模块划分

系统新增以下 Deep Module，每个模块具有简洁的接口、可独立测试：

模块	职责	接口复杂度	变更频率预估
`LLMClient`	封装 LLM API 调用，统一处理	低（3 个方法）	低
`ResumeParser`	接收文档文件/文本，输出结构	低（1 个公开方法）	中
`JDTranslator`	接收中文 JD，输出多语言 JD	低（1 个公开方法）	低
`MatchingEngine`	接收结构化简历 + JD，输出匹	低（1 个公开方法）	中
`ResumeOptimizer`	接收结构化简历 + JD + 差距	低（1 个公开方法）	中
`EmbeddingService`	封装 Embedding 模型调用，文	低（2 个方法）	低
`PromptManager`	管理所有 AI 调用的 Prompt	低（2 个方法）	中
`CostMonitor`	记录所有 LLM 调用的 Token 量	低（2 个方法）	低

模块接口契约

LLMClient

```
interface LLMClient {
  /**
   * 标准 LLM 调用，支持 Function Calling
   * @throws LLMLTimeoutError - 超过 30 秒未响应
   * @throws LLMLRateLimitError - API 限流
   * @throws LLMLFormatError - 返回格式不符合预期
   */
  complete(params: {
    model: 'fast' | 'powerful'
    messages: ChatMessage[]
    functions?: FunctionDefinition[]
    temperature?: number
  }): Promise<LLMResponse>

  /**
   * 流式调用，用于需要实时展示的场景
   */
}
```

```

streamComplete(params: {
  model: 'fast' | 'powerful'
  messages: ChatMessage[]
}): AsyncGenerator<string>

/**
 * 获取当前 Provider 的调用统计
 */
getStats(): LLMUsageStats
}

interface LLMResponse {
  content: string
  functionCall?: { name: string; arguments: Record<string, unknown> }
  usage: { promptTokens: number; completionTokens: number; totalCost: number }
  model: string
  latency: number
}

interface LLMUsageStats {
  todayTokens: number
  todayCost: number
  monthTokens: number
  monthCost: number
  budgetLimit: number
}

```

ResumeParser

```

interface ResumeParser {
  /**
   * 解析简历文档，返回结构化数据
   * 支持格式: PDF, DOCX, TXT, PNG/JPG (via OCR)
   * 支持语言: en, fr, es, ru, ar, zh, ja, ko
   */
  parse(input: {
    fileBuffer: Buffer
    fileName: string
    mimeType: string
    sourceLanguage?: string // 自动检测如果不提供
  }): Promise<ParsedResume>
}

interface ParsedResume {
  candidateId: string
  parsedAt: string
  sourceLanguage: string
  personalInfo: {
    name: string
    email: string | null
    phone: string | null
    nationality: string | null
    currentLocation: string | null
    languages: LanguageProficiency[]
  }
  education: EducationEntry[]
}

```

```

workExperience: WorkExperienceEntry[]
skills: string[] // 标准化为英文技能标签
certifications: string[]
intent: {
  desiredRole: string | null
  preferredIndustry: string[]
  availableFrom: string | null
  durationMonths: number | null
  locationPreference: string[]
}
rawText: string
confidence: number // 0-1, 整体解析置信度
}

interface LanguageProficiency {
  language: string
  level: 'native' | 'fluent' | 'advanced' | 'intermediate' | 'basic'
}

```

JDTranslator

```

interface JDTranslator {
  /**
   * 翻译 JD 到目标语言
   * 内部维护术语词典, 优先使用术语匹配
   */
  translate(input: {
    jdText: string
    jdStructure?: { responsibilities: string; requirements: string; benefits: string }
    targetLanguages: string[] // e.g. ['en', 'ja']
    companyIndustry?: string // 用于术语选择上下文
  }): Promise<TranslatedJD>
}

interface TranslatedJD {
  originalLanguage: string
  translations: Record<string, {
    title: string
    responsibilities: string
    requirements: string
    benefits: string
    culturalNotes: string[] // 中国特色词汇的解释
  }>
}

```

MatchingEngine

```

interface MatchingEngine {
  /**
   * 二阶段匹配: Embedding 粗排 → LLM 精排
   */
  match(params: {
    candidate: ParsedResume
    jobDescription: string
    maxResults?: number // 默认 20 (粗排), 最终返回 5 (精排)
  }): Promise<MatchResult>
}

```

```

/**
 * 批量匹配: 一个候选人 vs 多个 JD
 * 内部智能缓存 JD 向量
 */
batchMatch(params: {
  candidate: ParsedResume
  jobDescriptions: string[]
  maxResults?: number
}): Promise<MatchResult[]>
}

interface MatchResult {
  overallScore: number // 0-100
  dimensionScores: {
    skillMatch: number
    experienceMatch: number
    educationMatch: number
    intentMatch: number
    languageMatch: number
  }
  summary: string // LLM 生成的中文总结
  highlights: string[] // 匹配亮点
  gaps: string[] // 差距分析
  confidence: number // 0-1
}

```

ResumeOptimizer

```

interface ResumeOptimizer {
  /**
   * 生成简历优化建议
   */
  optimize(params: {
    resume: ParsedResume
    targetJob: string // JD 文本
    gapAnalysis: string[] // 来自 MatchingEngine 的 gaps
  }): Promise<OptimizationSuggestion[]>
}

interface OptimizationSuggestion {
  id: string
  category: 'skill' | 'description' | 'keyword' | 'structure' | 'culture'
  priority: number // 1-10, 10 为最高优先级
  title: string
  currentState: string
  suggestedChange: string
  whyItMatters: string
  estimatedImpact: number // 预估匹配度提升分数
}

```

EmbeddingService

```

interface EmbeddingService {
  /**
   * 生成文本的向量表示

```

```

*/
embed(text: string): Promise<number[]>

/**
 * 批量向量化
 */
batchEmbed(texts: string[]): Promise<number[][]>
}

```

数据库 Schema 设计

— 候选人结构化简历

```

CREATE TABLE candidates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
  name TEXT NOT NULL,
  email TEXT,
  phone TEXT,
  nationality TEXT,
  current_location TEXT,
  source_language TEXT NOT NULL,
  raw_text TEXT NOT NULL,
  parsed_data JSONB NOT NULL, -- 完整 ParsedResume 的 JSON
  confidence REAL DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 简历向量 (用于 RAG 检索)
CREATE TABLE resume_vectors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  candidate_id UUID NOT NULL REFERENCES candidates(id) ON DELETE CASCADE,
  embedding VECTOR(3072) NOT NULL -- text-embedding-3-large 维度
);

CREATE INDEX ON resume_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

— 岗位

```

CREATE TABLE jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL,
  title_zh TEXT NOT NULL,
  title_en TEXT,
  jd_zh TEXT NOT NULL,
  jd_en TEXT,
  structured_jd JSONB, -- 结构化 JD (技能要求、学历要求等)
  status TEXT DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 岗位向量
CREATE TABLE job_vectors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id UUID NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  embedding VECTOR(3072) NOT NULL
);

CREATE INDEX ON job_vectors USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

— 匹配记录

```
CREATE TABLE matches (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  candidate_id UUID NOT NULL REFERENCES candidates(id),
  job_id UUID NOT NULL REFERENCES jobs(id),
  overall_score REAL NOT NULL,
  dimension_scores JSONB NOT NULL,
  summary TEXT,
  highlights TEXT[],
  gaps TEXT[],
  calculated_at TIMESTAMPTZ DEFAULT NOW(),
  user_feedback TEXT -- 'like' | 'dislike' | null
);
```

— API 调用成本记录

```
CREATE TABLE llm_cost_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  module TEXT NOT NULL, -- 'resume_parser' | 'jd_translator' | 'matching' | 'optimizer'
  model TEXT NOT NULL,
  prompt_tokens INT NOT NULL,
  completion_tokens INT NOT NULL,
  cost REAL NOT NULL,
  latency_ms INT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX ON llm_cost_logs (created_at);
CREATE INDEX ON llm_cost_logs (module);
```

关键架构决策

JD

向量缓存策略: JD

是可缓存资产，发布时生成一次向量，只有编辑时才重新生成。简历向量在候选人每次更新后重新生成。这使 80% 的匹配查询命中缓存。

模型分级路由: LLMClient 内部维护两个模型映射——fast 映射到 GPT-4o-mini / Claude Haiku, powerful 映射到 GPT-4o / Claude Sonnet。简历解析、匹配精排用 powerful，翻译、优化建议用 fast。

降级链: 当 primary LLM Provider 不可用时，自动切换到 secondary Provider。当所有 Provider 都不可用时，匹配引擎回退到纯 Embedding 余弦相似度排序（无 LLM 精排），翻译回退到缓存结果。

不引入消息队列: MVP 阶段所有 AI 调用同步完成。理由: 日活 500 时，峰值 QPS < 5，同步调用完全可以接受。Phase 4 引入队列处理批量匹配。

Testing Decisions

测试原则

只测试模块的外部行为（输入/输出），不测试实现细节

所有 LLM 调用在测试中用 mock 替代，避免测试依赖外部 API

每个 Deep Module 应有独立的单元测试文件

各模块测试策略

模块	测试重点	Mock 策略
`LLMClient`	正常响应、超时重试、Provider 切换、限流	Mock HTTP 层
`ResumeParser`	英文简历解析准确率、多语言识别、OCR	Mock LLM 返回固定结构化数据
`JDTranslator`	术语翻译准确率、文化词汇标注、结构化	Mock LLM 返回固定翻译结果
`MatchingEngine`	粗排召回率、精排评分合理性、缓存命中	Mock Embedding 返回固定向量，Mock LLM 返回固定分数
`ResumeOptimizer`	建议数量、优先级排序、影响力预估	Mock LLM 返回固定建议列表
`EmbeddingService`	向量维度正确性、批量处理	Mock API 返回固定维度向量
`PromptManager`	模板版本切换、变量注入正确性	直接测试，无需 mock
`CostMonitor`	预算告警触发、日报统计	直接测试

API 集成测试

使用 curl 或 supertest 对以下端点编写 smoke test:

POST /api/resume/parse — 上传示例英文 PDF，验证返回结构化 JSON

POST /api/jd/translate — 提交中文 JD，验证返回英文版

POST /api/match — 提交简历 ID + 岗位 ID，验证返回匹配分数和理由

POST /api/resume/optimize — 提交简历 + 目标 JD，验证返回 ≥ 3 条有效建议

Out of Scope

以下功能明确不在本 PRD 范围内:

自训练或微调模型: MVP 阶段完全依赖 LLM API，不进行任何模型训练

视频面试功能

Further Notes

Phase 4: 商业化与规模化（远期规划）

在 Phase 1-3 完成后（预计上线后 3-6 个月），启动 Phase 4:

任务	说明
付费订阅模型	企业端: 免费版每月 5 次 AI 匹配 / 付费版无限匹配 + 批量导入。学生端:
K8s 迁移	从单机 Docker Compose 迁移到 K8s 集群，支持水平扩缩容
Milvus 迁移	向量数据 > 100 万时从 pgvector 迁移到 Milvus，支持 ANN 索引和分布式检

异步匹配队列	引入 BullMQ + Redis, 企业批量导入简历时异步处理匹配
A/B 实验框架	支持 Prompt 版本 A/B 测试, 用匹配转化率评估不同 Prompt 的效果
多语言 JD 扩展	在英文基础上增加日语、韩语、越南语翻译支持
简历模板市场	提供面向中国企业的简历模板, 国际学生可一键套用
企业画像	基于历史招聘数据构建企业偏好画像, 提高匹配精准度

成本估算（月度，日活 500）

模块	月调用量	模型	预估月成本
简历解析	2,000 次	GPT-4o	800
岗位翻译	500 次	GPT-4o-mini	50
人岗匹配	10,000 次	Embedding + GPT-4o-mini	600
简历优化	3,000 次	GPT-4o-mini	150
合计			**1,600**

实际成本预计低于此估算，因缓存率高（翻译和 JD 向量几乎 100% 命中缓存）。